

Institut National de Radioélectricité et de Cinématographie

Enseignement technique secondaire de qualification

Accès aux études supérieures



Avenue Jupiter, 188
1190 Forest

Conception d'une messagerie instantanée chiffrée
« *Nexuschat* »

Projet personnel de Derick, Maxime
Pour l'obtention du certificat de qualification

Tuteur : Abdelkader Ben Sellam
Année scolaire : 2024-2025

Table des matières

1. Introduction	5
1.1. Topologie	6
1.1.1 Description de l'architecture réseau.....	7
1.1.1.1 Infrastructure locale	7
1.1.1.2 Infrastructure distante	7
1.1.1.3 Accès sécurisé.....	8
1.1.1.3.1 Accès utilisateur	8
1.1.1.3.2 Accès administrateur	8
1.1.1.4 Conclusion	8
2. Matériels et logiciels.....	9
3. Mise en place de l'infrastructure Locale	12
3.1. Proxmox.....	12
3.2. Debian	12
3.3. Machine virtuelle 1	13
3.3.1. MariaDB.....	13
3.3.2. Tables	14
3.3.2.1. data_users	14
3.3.2.2. Conversations	14
3.3.2.3. Contacts	15
3.3.2.4. Messages.....	15
3.3.2.5. Demandes.....	16
3.3.2.6. notifications.....	16
3.3.3. Configuration fichier MariaDB.....	17
3.3.4. Utilisateur SQL.....	18
3.3.5. Openssl.....	19
3.4. Machine virtuelle 2.....	20
3.4.1. Serveur Docker.....	20
3.4.2. Apache2.....	20
3.4.3. WireGuard	20
3.5. Raspberry	21
3.5.1. Python.....	21
3.5.1.1. Mise en place des API	22
3.5.1.1.1. Proftpd	23
3.5.1.1.1.1. Configuration proftpd	23
3.5.1.1.1.2. Connexion via FileZilla	24
3.5.1.2. PyMySQL.....	25
3.5.1.3. Bcrypt	26
3.5.1.4. Nacl.....	26
3.5.1.5. Smtplib	27
3.5.2. Let's encrypt.....	28
3.5.3. Domain Name Server (DNS).....	29
3.5.4. Redirections de ports.....	30
3. Mise en place de l'infrastructure distante.....	31
3.5. Flutter	31
3.6. Interface graphique	32
3.7. Android (APK)	32

3.7.5.	Dropbox	32
4.	<i>Ce que j'aurais aimé mettre en place.....</i>	33
5.2.	Ajout d'un service volP	33
5.3.	possibilité d'envoyer des médias.....	33
5.4.	Correction de bugs et améliorations.....	34
5.5	Système de monitoring.....	34
5.	<i>Conclusion.....</i>	35
6.	<i>Sitographie</i>	36
5.1.	Apache2	36
5.2.	Let's encrypt.....	36
5.3.	MariaDB.....	36
5.4.	Openssl	37
5.5.	Proxmox.....	37
5.6.	Proftpd	37
5.7.	Pymysql	38
5.8.	python	38
5.8.1.	FastAPI	38
5.8.2.	Bcrypt.....	38
5.8.3.	Nacl.....	38
5.8.4.	smtplib	39
5.9.	Flutter	39
5.10.	Intelligence artificielle.....	39
5.11.	Raspberry	39
6.	<i>Annexe</i>	41
6.1.	Ordinateur hébergeant Proxmox	41
6.2.	Raspberry Pi 5.....	42
6.3.	Hashage de mot de passe	42
6.4.	API-FastAPI.....	43
6.5.	Docker	44
6.6.	Email de Réinitialisation de mot de passe.....	45
6.7.	Email de tentative de connexion.....	46
6.8.	Email de confirmation de compte	47
6.9.	Interface utilisateur.....	48
6.9.1.	Page de connexion	48
6.9.2.	Page d'inscription.....	49
6.9.3.	Liste des messages	50
6.9.4.	Profil de l'utilisateur.....	51
6.9.5.	Page de contact (liste de contacts de l'utilisateur).....	52
6.9.6.	Page de contact section recherche	52
6.9.7.	Page de notifications	53

6.10.	Interface administrateur.....	54
--------------	--------------------------------------	-----------

1. Introduction

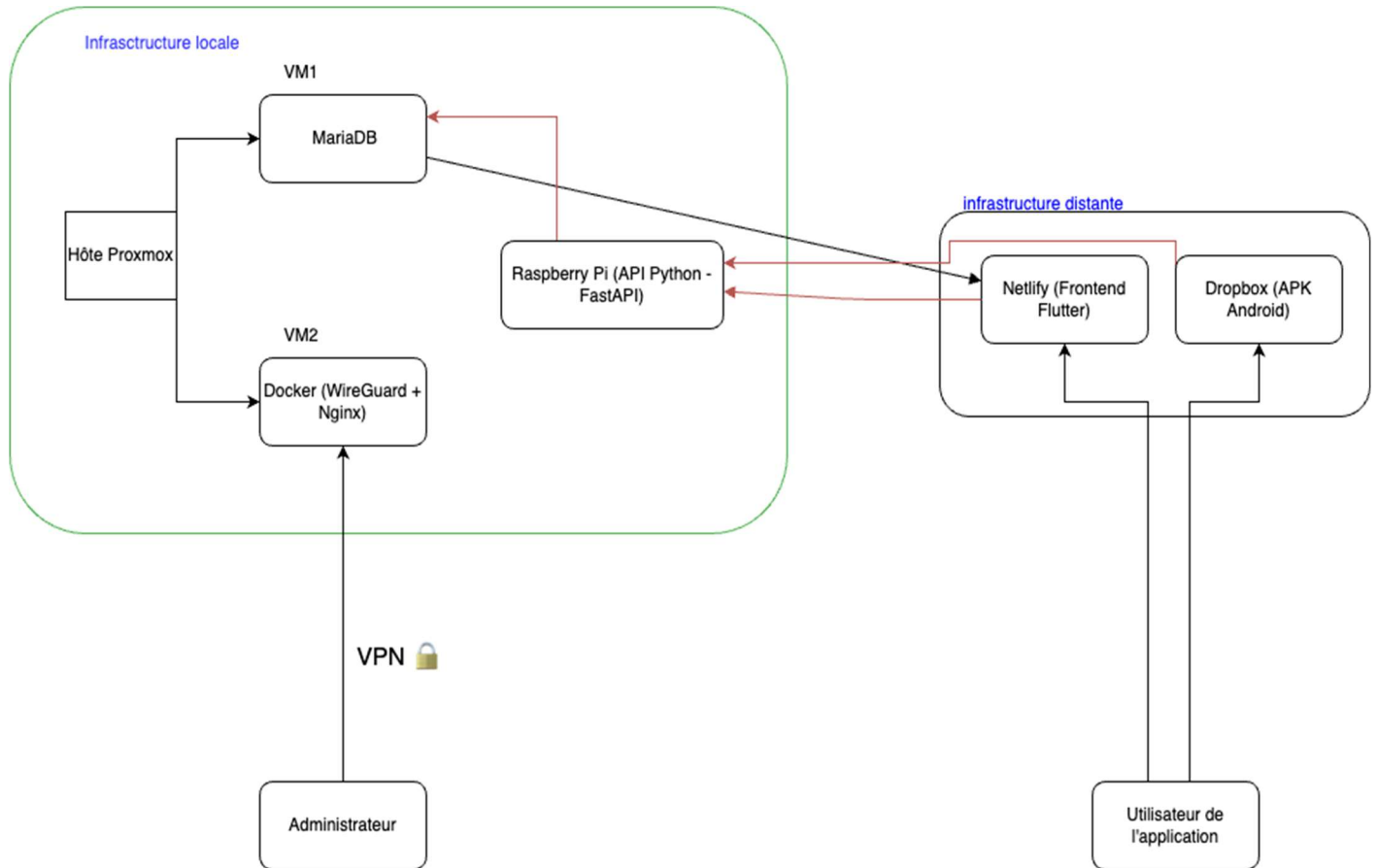
La communication à travers nos appareils électroniques est devenue indispensable dans notre vie quotidienne. Il s'agisse d'un message banal envoyé pour souhaiter un bon anniversaire ou d'un document stratégique envoyé à un collaborateur, ces gestes anodins sont pourtant de véritables occasions offertes à des personnes aux intentions malveillantes.

De même, la plupart des utilisateurs utilisent aujourd'hui Messenger, WhatsApp ou Instagram pour échanger avec leurs proches. Or, ces applications appartiennent à la même entreprise : Meta :

centraliser autant de données personnelles dans une seule entreprise pose aussi des questions au sujet de la confidentialité et de la sécurité. Qui a accès à nos échanges ? Que deviennent nos données échangées ? Comment garantir qu'elles ne tombent pas entre de mauvaises mains lorsque nous avons une cyberattaques aussi dérespectueuse ?

Dans un monde où le cybercriminalité augmente et où l'internalisation va croissant, se préoccuper de moyens de communication plus sûrs devient une urgence : d'où l'idée de cheminer vers un chat instantané, et chiffré de bout à bout.

1.1. Topologie



1.1.1 Description de l'architecture réseau

L'infrastructure déployée repose sur une séparation claire entre une infrastructure locale et une infrastructure distante, dans un souci de sécurité, de modularité et de performance.

1.1.1.1 *Infrastructure locale*

Au cœur de l'infrastructure locale se trouve un hôte Proxmox, utilisé comme hyperviseur pour gérer plusieurs machines virtuelles. Deux VMs principales y sont déployées :

- VM1 – MariaDB : Cette machine héberge un serveur de base de données MariaDB, responsables des données liés à l'application.
- VM2 – Docker (WireGuard + Apache2) : Cette seconde VM héberge un environnement Docker, dans lequel sont exécutés les services d'infrastructure tels que le serveur VPN WireGuard et Apache2, utilisés pour sécuriser accéder à une interface d'administration.

Un Raspberry Pi, rattaché au réseau local, exécute des API, que j'ai développées en Python via FastAPI. Cette API est le point d'entrée pour les échanges de données avec la base MariaDB, centralisant la logique, et fait le pont entre la base de données et les utilisateurs (par requêtes directes de la base), tout en préservant l'isolement du serveur principal de gestion de l'hyperviseur.

1.1.1.2 *Infrastructure distante*

Le frontend de l'application est développé avec Flutter et déployé sur Netlify, une plateforme de déploiement moderne pour sites web statiques. Ce front-end communique avec l'API FastAPI hébergée localement via une connexion sécurisée, lui permettant d'offrir à l'utilisateur final une interface fluide et responsive.

1.1.1.3. Accès sécurisé

1.1.1.3.1. Accès utilisateur

La façon dont l'utilisateur va se connecter est une connexion par HTTPS. Ce qui permet de garantir la confidentialités des données lors de l'échange entre client et serveurs. Grâce à l'utilisation des certificats SSL/TLS la sécurité de la transmission est garantie.

1.1.1.3.2. Accès administrateur

L'accès réservé à l'administrateur est davantage restreint et s'effectue via un VPN configuré avec WireGuard. Ce réseau privé virtuel permet d'établir une connexion chiffrée entre l'administrateur et le serveur, rendant possible l'accès à une interface d'administration hébergée en local, qui est inaccessible depuis Internet. Ainsi, seules les personnes disposant des bonnes clés VPN peuvent administrer le système à distance de manière sécurisée (voir annexe).

1.1.1.4. Conclusion

Cette topologie montre les avantages de la virtualisation locale, de l'API intermédiaire, de la séparation front/back et d'un modèle de sécurité basé sur le cloisonnement et l'accès restreint. Ce type d'organisation est particulièrement pertinent pour un projet de messagerie sécurisée ou toute application nécessitant un contrôle rigoureux de la circulation des données.

2. Matériels et logiciels

Pour concevoir ce projet, j'ai dû réunir différents outils matériels et logiciels, chacun jouant un rôle précis et essentiel dans son bon fonctionnement. Sans ces éléments, le système mis en place ne pourrait tout simplement pas fonctionner correctement. L'objectif est de démontrer qu'il est possible de mettre en place une solution de communication sécurisée, accessible et respectueuse de la vie privée des utilisateurs.

Nom	Type	Utilité	prix
ordinateur	Matériel	Serveur principal pour héberger les services (base de données, FTP, etc.)	200€ (prix d'un ordinateur d'occasion de moyenne gamme)
Raspberry Pi 5	Matériel	Le raspberry va me servir d'hébergeur à mes API c'est-à-dire que pour communiquer avec ma base de données mon application va communiqué avec mon raspberry	97€
Python	Logiciel	Langage principal pour gérer la communication avec la base de données	Gratuit
Dart	Logiciel	Utilisé pour concevoir l'interface utilisateur de l'application de manière moderne et agréable à utiliser	Gratuit

MariaDB	Logiciel	Gestionnaire de base de données pour stocker les utilisateurs et messages, les conversations, les clés de chiffrement des messages.	Gratuit
Let's encrypt	Logiciel	Fournisseur de certificat ssl pour les requêtes https	Gratuit
Debian	Logiciel	Système d'exploitation open source, stable et sécurisé, utilisé pour héberger le projet	Gratuit
Docker	Logiciel	Docker a été utilisé dans ce projet pour créer des environnements isolés, légers et facilement déployables. Dans mon cas un vpn et un environnement apache2	Gratuit
Wireguard	Logiciel	Celui-ci me permet d'accéder à l'ensemble des services du projet sans exposer les ports sensibles directement sur Internet, assurant ainsi une couche de sécurité supplémentaire.	Gratuit

DNS (facultatif)	Logiciel	Au lieu de mettre à chaque fois l'ip publique de mon modem j'ai décidé d'utiliser le dns fournit lors de l'achat de mon nom de domaine pour avoir plus facile lors de l'insertion des liens API	14€
Openssl	Logiciel	Pour m'assurer que la communication soit chiffré entre le serveur hébergeant la base de données et le raspberry j'utilise des certificats SSL	Gratuit

3. Mise en place de l'infrastructure Locale

Avant de commencer à développer ou configurer quoi que ce soit, j'ai du mettre en place une infrastructure pour poser une base solide à mon projet afin de pouvoir héberger les différents services nécessaires au bon fonctionnement de ma messagerie.

3.1. Proxmox

Proxmox est un environnement de virtualisation open source. Il permet de créer, gérer et exécuter plusieurs machines virtuelles (VM) sur un seul serveur physique. Chaque VM peut être configurée pour un usage spécifique : base de données, serveur FTP, serveur web, etc. Ce découpage me permet d'isoler les services, de mieux organiser mon infrastructure et d'intervenir plus facilement en cas de problème.

3.2. Debian

Pour les services que je vais déployer le système d'exploitation que j'ai choisi est Debian. Cette distribution de linux est open-source et une des plus stable et documenté. Ce qui me permet de déployer des services sécurisé et stable et évidemment de la maintenir régulièrement à jour.

Au total, j'ai mis en place trois machines virtuelles sous Debian, chacune ayant un rôle spécifique dans le bon déroulement de mon projet :

1. Serveur de base de données : héberge les données liées aux utilisateurs et aux messages.
2. Serveur Docker : contient les conteneurs nécessaires (WireGuard, Apache2).
3. Environnement de test : me permet de tester des configurations ou des mises à jour sans impacter le système principal.

3.3. Machine virtuelle 1

3.3.1. MariaDB

Toujours dans une logique de solutions open source, j'ai choisi MariaDB pour la gestion de ma base de données. Ce système est une alternative à MySQL, largement utilisée, bien documentée et reconnue pour sa simplicité d'utilisation et sa fiabilité. Grâce à MariaDB, je peux stocker et gérer les informations des utilisateurs, les messages échangés, ainsi que d'autres données essentielles à mon application.

Évidemment, il ne suffit pas de stocker des données ; il faut aussi les organiser de manière logique et efficace pour pouvoir y accéder et interagir avec aisément. Pour cela, j'ai conçu plusieurs tables, chacune ayant un rôle bien défini dans la structure de ma base de données. Cette organisation permet de maintenir la cohérence des données, de faciliter les requêtes et de gérer les relations entre les différents éléments du système.

Voici un aperçu des principales tables utilisées :

- `data_users` : Table essentielle du projet, elle enregistre les données des utilisateurs telles que l'identifiant, l'adresse e-mail, le nom d'utilisateur, le mot de passe (haché), ainsi que la date de création du compte.
- `conversations` : Elle permet de gérer les conversations entre utilisateurs, en identifiant qui communique avec qui.
- `contacts` : Cette table stocke les relations établies entre les utilisateurs, c'est-à-dire leur liste de contacts validés.
- `demandes` : Elle enregistre les demandes de contact envoyées entre utilisateurs, en attente d'approbation.
- `messages` : Elle contient les messages échangés, en indiquant l'expéditeur, le destinataire, le contenu du message et la date d'envoi.
- `Notifications` : Cette table contient les notifications de chaque utilisateurs afin de prévenir l'utilisateur des nouveaux messages.

3.3.2. Tables

3.3.2.1. data_users

Contient les informations principales des utilisateurs de l'application.

- id : identifiant unique de l'utilisateur (clé primaire).
- date_creation : date et heure de création du compte.
- username : nom d'utilisateur unique (login).
- password : mot de passe hashé de l'utilisateur.
- email : adresse e-mail utilisée pour l'inscription.
- email_verify : indique si l'e-mail a été vérifié (1 = oui, 0 = non).

Field	Type	Null	Key	Default	Extra
id	int(11)	NO	PRI	NULL	auto_increment
date_creation	timestamp	YES		current_timestamp()	
username	varchar(11)	YES	UNI	NULL	
password	varchar(255)	YES		NULL	
email	varchar(50)	NO		NULL	
email_verify	tinyint(1)	YES		NULL	

3.3.2.2. Conversations

Enregistre chaque conversation entre deux utilisateurs.

- id : identifiant unique de la conversation.
- user1_username : premier utilisateur impliqué dans la conversation.
- user2_username : deuxième utilisateur.
- created_at : date de création de la conversation.

Field	Type	Null	Key	Default	Extra
id	int(11)	NO	PRI	NULL	auto_increment
user1_username	varchar(255)	NO	MUL	NULL	
user2_username	varchar(255)	NO	MUL	NULL	
created_at	timestamp	YES		current_timestamp()	

3.3.2.3. Contacts

Stocke les contacts validés pour chaque utilisateur.

- id : identifiant unique de la relation.
- owner : nom de l'utilisateur propriétaire de la liste de contacts.
- contacts : nom de l'utilisateur ajouté comme contact.
- ad_at : date d'ajout du contact.

Field	Type	Null	Key	Default	Extra
id	int(11)	NO	PRI	NULL	auto_increment
owner	varchar(25)	YES		NULL	
contacts	varchar(25)	YES		NULL	
ad_at	timestamp	YES		current_timestamp()	

3.3.2.4. Messages

Contient tous les messages échangés entre utilisateurs.

- id : identifiant unique du message.
- expéditeur : nom d'utilisateur de l'expéditeur.
- destinataire : nom d'utilisateur du destinataire.
- id_conversation : identifiant de la conversation liée.
- messages : contenu du message (chiffré).
- sent_at : date et heure d'envoi.
- Type : va permettre d'identifier si le message est un texte, un gif ,une photo pour pouvoir l'afficher correctement avec Flutter.
- key : clé de chiffrement utilisée pour ce message

Field	Type	Null	Key	Default	Extra
id	int(11)	NO	PRI	NULL	auto_increment
expediteur	varchar(12)	NO		NULL	
destinataire	varchar(12)	NO		NULL	
id_conversation	int(12)	NO		NULL	
messages	varchar(255)	NO		NULL	
sent_at	timestamp	YES		current_timestamp()	
type	varchar(255)	NO		NULL	
key	varchar(255)	YES		NULL	

3.3.2.5. Demandes

Gère les demandes de contact envoyées entre utilisateurs.

- id : identifiant unique de la demande.
- owner : utilisateur qui a reçu la demande.
- from : utilisateur qui a envoyé la demande.
- sent_at : date de l'envoi de la demande.
- status : état actuel de la demande (pending, accepted, rejected).

Field	Type	Null	Key	Default	Extra
id	int(11)	NO	PRI	NULL	auto_increment
owner	varchar(25)	YES		NULL	
from	varchar(25)	YES		NULL	
sent_at	timestamp	YES		current_timestamp()	
status	enum('pending','accepted','rejected')	YES		pending	

3.3.2.6. notifications

- id : identifiant unique de la notification.
- destinataire : utilisateur a qui est adressée la notification
- type : le type est l'objet de la notification.
- contenu : le contenu de la notification (le message ou autre)
- date_creation : date à laquelle à été envoyé la notification.

Field	Type	Null	Key	Default	Extra
id	int(11)	NO	PRI	NULL	auto_increment
destinataire	varchar(255)	NO		NULL	
type	varchar(50)	NO		NULL	
contenu	text	NO		NULL	
date_creation	datetime	YES		current_timestamp()	

+-----+-----+-----+-----+-----+

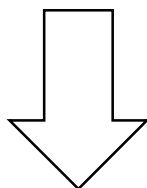
3.3.3. Configuration fichier MariaDB

Par défaut, MariaDB est configuré pour n'écouter que les connexions provenant de la machine locale (loopback), via l'adresse IP 127.0.0.1. Cela signifie que les utilisateurs situés en dehors du serveur — que ce soit sur le réseau local (LAN) ou sur un réseau étendu (WAN) — ne peuvent pas accéder à la base de données.

Pour faire cela la modification du fichier qui se trouve dans le répertoire
/etc/mysql/mariadb.conf.d/50-server.cnf est nécessaire

Une fois dans le fichier la ligne « bind-address = «127.0.0.1 » doit être modifiée en bind-address
= 0.0.0.0

```
# localhost which is more compatible and  
bind-address          = 127.0.0.1
```



```
# localhost which is more compatible and is n  
bind-address          = 0.0.0.0
```

3.3.4. Utilisateur SQL

Pour permettre la connexion à la base de données, la création d'un utilisateur spécifique est nécessaire. Cet utilisateur disposera des droits nécessaires pour interagir avec les tables de l'application, tout en garantissant un niveau de sécurité adapté.

Voici la création de l'utilisateur maxime qui est un des administrateurs de l'application donc nous lui avons attribué tout les privilèges

```
MariaDB [(none)]> create user 'maxime'@'%' identified by "Inraci#2024";  
Query OK, 0 rows affected (0.058 sec)
```

Le symbole % dans 'maxime'@'%' permet à l'utilisateur de se connecter depuis n'importe quelle adresse IP

```
MariaDB [(none)]> grant all privileges on *.* to 'maxime'@'%';  
Query OK, 0 rows affected (0.001 sec)
```

La commande flush privileges recharge les droits accordés sans redémarrer le serveur.

```
MariaDB [(none)]> flush privileges;  
Query OK, 0 rows affected (0.000 sec)
```

Et enfin bien que l'utilisateur maxime existe et qui peut se connecter via n'importe quelle adresse

```
MariaDB [(none)]> select user,host from mysql.user;
```

```
+-----+-----+  
| User   | Host   |  
+-----+-----+  
| maxime | %      |  
| mariadb.sys | localhost |  
| mysql  | localhost |
```

| root | localhost |

+-----+-----+

3.3.5. Openssl

Garantir la confidentialité de chaque données échangées entre la base de données et l'API est quelque chose de primordial sans cela le projet n'a plus de sens car n'importe qui qui captura le paquet aura accès à la donnée transmise.

Pour palier à cela j'ai auto généré des certificats ssl auto signé. Cela va permettre d'avoir une liaison chiffré et sécurisée.

Les étapes pour créer et appliqué des certificats ssl auto-signés à un serveur mariadb:

1. Générer une clé privée pour la CA :

```
openssl genrsa -out Nexuschat_authority.key 4096
```

2. Créer un certificat racine auto-signé pour la CA :

```
openssl req -x509 -new -nodes -key Nexuschat_authority.key -sha256 -days 3650 -out  
Nexuschat_authority.pem
```

3. Créer une clé privée pour le serveur :

```
openssl genrsa -out serveur.key 2048
```

4. Créer une demande de certificat pour le serveur :

```
openssl req -new -key serveur.key -out serveur.csr
```

5. Signer le certificat serveur avec votre CA :

```
openssl x509 -req -in serveur.csr -CA Nexuschat_authority.pem -CAkey Nexuschat_authority.key -  
CAcreateserial -out  
serveur.crt -days 825 -sha256
```

6. Appliquer les certificats :

Dans le fichier de configuration /etc/mysql/mariadb.conf.d/50-server.cnf, Ajoute ou modifie les lignes suivantes dans la section [mysqld] :

ssl-ca = /etc/mysql/certs/Nexuschat_authority.pem

ssl-cert = /etc/mysql/certs/serveur.crt

ssl-key = /etc/mysql/certs/serveur.key

require_secure_transport = ON

3.4. Machine virtuelle 2

3.4.1. *Serveur Docker*

J'ai décidé de recourir à Docker, un outil open source largement utilisé dans les milieux du développement et de l'administration système. Docker permet de créer des conteneurs, c'est-à-dire des environnements isolés et légers, capables de faire tourner des applications ou des services comme s'ils fonctionnaient sur un système dédié.

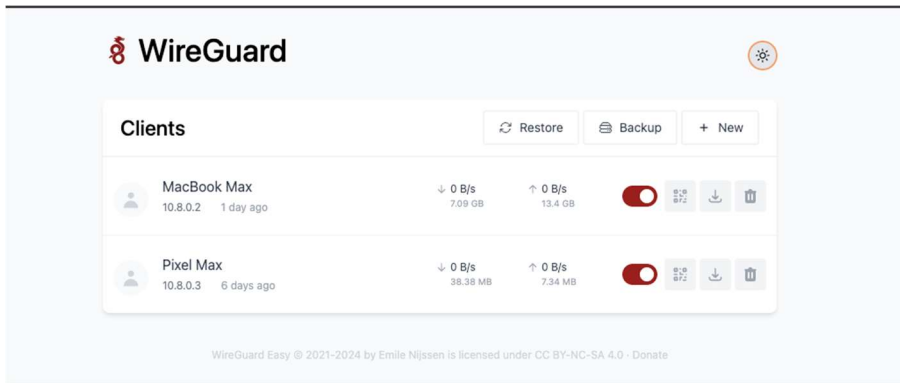
3.4.2. *Apache2*

Dans mon projet, j'utilise Apache2 pour héberger un site web d'administration de la base de données. Ce site permet une gestion plus intuitive de MariaDB, notamment dans le cas où un autre administrateur, peu familier avec le terminal ou avec la syntaxe SQL, devrait reprendre le projet. Grâce à cette interface graphique, la prise en main de la base devient beaucoup plus accessible.

3.4.3. *WireGuard*

Un VPN (via WireGuard) est également déployé dans un conteneur Docker. Il permet aux administrateurs d'accéder à ce site de manière sécurisée, tout en évitant de rendre le service accessible publiquement sur Internet. En obligeant le passage par une connexion VPN privée, j'ajoute une couche de sécurité importante tout en gardant une configuration simple à maintenir.

Actuellement, étant le seul administrateur de l'application, seuls deux appareils sont autorisés à accéder au réseau hébergeant mes machines : mon ordinateur et mon smartphone. Ces deux dispositifs me permettent d'administrer l'infrastructure où que je sois.



3.5. Raspberry

En ce qui concerne le Raspberry Pi, j'ai opté pour l'installation du système d'exploitation Ubuntu. Réputé pour sa documentation abondante, sa communauté dynamique et sa stabilité éprouvée en environnement serveur, Ubuntu s'est rapidement imposé comme le choix le plus pertinent pour ce type de matériel.

Le Raspberry Pi joue ici un rôle essentiel : il sert d'hébergeur pour mes API Python développées avec FastAPI. Grâce à Ubuntu, je bénéficie d'un système à la fois léger, fiable et facile à maintenir, parfaitement adapté à une utilisation en tant que serveur applicatif dans une infrastructure personnelle.

3.5.1. Python

Comme mentionné précédemment, mes API sont conçues avec FastAPI, une bibliothèque moderne du langage Python. C'est un langage de programmation puissant, très utilisé dans le développement backend pour sa syntaxe claire, sa large communauté et ses bibliothèques fournies.

Pour développer les API de mon application, j'ai adopté le style architectural REST (Representational State Transfer). REST est un ensemble de principes définis pour structurer des services web. Il repose sur l'utilisation des méthodes HTTP standards (GET, POST, PUT, DELETE, etc.) pour interagir avec des ressources clairement identifiées via des URLs (appelées URI).

Une API REST suit plusieurs règles fondamentales :

- Elle est stateless, c'est-à-dire que le serveur ne conserve pas l'état des interactions : chaque requête contient toutes les informations nécessaires à son traitement.
- Elle utilise une interface uniforme, avec des conventions d'accès communes à toutes les ressources.
- Elle retourne des représentations des données, souvent au format JSON, permettant au client de les consommer facilement.
- Elle peut être mise en cache pour optimiser les performances.

Dans mon cas, les différentes entités (utilisateurs, messages, contacts, etc.) sont accessibles via des routes bien définies, en suivant les conventions REST. Cela rend l'API claire, lisible et compatible avec la majorité des clients web ou mobiles.

Grâce à cette approche, l'API de Nexuschat bénéficie d'une architecture rigoureuse, d'une exécution rapide d'une bonne évolutivité, ce qui en fait une solution solide pour une messagerie sécurisée.

3.5.1.1. Mise en place des API

Une fois les API prêtes à être utilisées, il me fallait mettre en place un moyen simple, rapide et efficace de les transférer sur le Raspberry Pi. Il était essentiel de pouvoir les déployer facilement, mais aussi de pouvoir les mettre à jour régulièrement au fur et à mesure de l'évolution du projet.

Pour répondre à ce besoin, j'ai choisi de configurer le Raspberry Pi comme un serveur FTP (File Transfer Protocol). Cette solution me permet de me connecter à distance et de transférer directement les fichiers Python contenant les API, que ce soit pour une première installation ou pour des modifications ultérieures.

Ainsi, l'utilisation d'un serveur FTP offre une grande souplesse dans la gestion du code et me permet de maintenir les API à jour sans avoir à intervenir physiquement sur la machine.

3.5.1.1.1. Proftpd

Dans le cas où une mise à jour de l'application serait nécessaire, j'ai mis en place un serveur ProFTPD. Ce serveur permet d'accéder directement aux fichiers python de l'API, facilitant ainsi le déploiement, la modification ou le remplacement des API sans devoir intervenir physiquement sur la machine. Grâce à cette solution, les mises à jour peuvent être effectuées de manière simple, rapide et à distance.

Pour le mettre en place, j'ai d'abord modifié le fichier proftpd.conf situé dans le répertoire /etc/proftpd. J'ai ajouté des droits d'accès au répertoire, permettant la lecture et l'écriture aux utilisateurs autorisés à se connecter au serveur ProFTPD.

J'ai également décidé que la gestion de l'interface se ferait avec l'utilisateur derick. Ainsi, à l'aide de la directive DefaultRoot, j'ai restreint cet utilisateur au répertoire /derick/API, pour des raisons de praticité.

3.5.1.1.1.1. Configuration proftpd

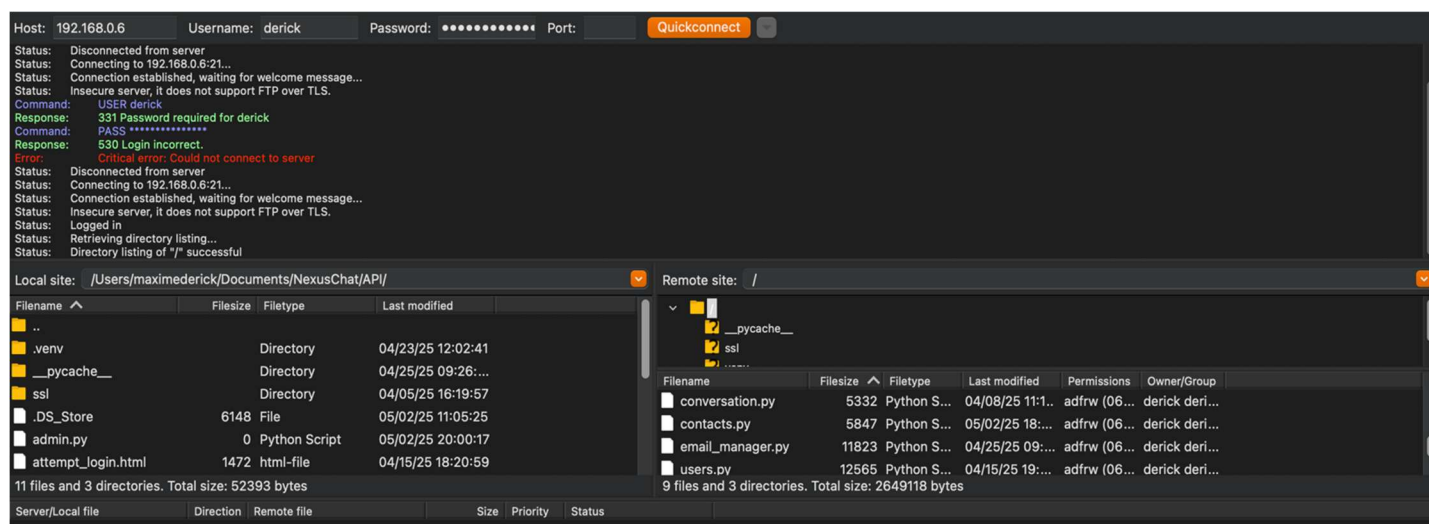
Voici ce les commandes implémenté dans le fichier proftpd.conf :

```
<Directory /home/derick/API>  
<Limit READ WRITE>  
AllowAll  
</Limit>  
</Directory>  
DefaultRoot /home/derick/API derick
```

3.5.1.1.1.2. Connexion via FileZilla

Pour gérer les fichiers plus facilement, j'utilise FileZilla, une application me permettant de me connecter en FTP via une interface graphique. Je préfère utiliser FileZilla car son interface intuitive rend la navigation dans les répertoires et la gestion des fichiers bien plus simple et rapide qu'en ligne de commande.

Grâce à cette interface, je peux glisser-déposer des fichiers depuis ma machine locale vers le serveur distant, ou inversement, sans avoir à taper de commandes. Cela me permet de gagner du temps, surtout lors de mises à jour fréquentes de l'application.



3.5.1.2. PyMySQL

PyMySQL, un module Python qui permet de se connecter et d'interagir avec une base de données MySQL ou MariaDB. Cette bibliothèque est essentielle au bon fonctionnement de l'application, car elle assure la communication entre le code Python (back-end) et la base de données.

PyMySQL permet d'effectuer les opérations suivantes :

- établir une connexion sécurisée à une base de données distante ou locale,
- exécuter des requêtes SQL (INSERT, SELECT, UPDATE, DELETE),
- récupérer les résultats des requêtes sous forme de données exploitables en Python,
- et gérer les transactions (validation ou annulation des changements en base).

Dans mon projet, PyMySQL est utilisé pour :

- enregistrer les utilisateurs lors de l'inscription,
- authentifier un utilisateur lors de la connexion,
- stocker et lire les messages chiffrés,
- et suivre l'activité des utilisateurs dans différentes tables.

Grâce à cette bibliothèque, l'application peut manipuler dynamiquement les données de la base tout en restant fluide et sécurisée.

3.5.1.3. Bcrypt

Bcrypt est une bibliothèque de hachage cryptographique utilisée pour sécuriser les mots de passe. Dans mon projet, je l'ai utilisée pour transformer les mots de passe des utilisateurs en chaînes illisibles avant de les stocker en base de données.

Le hachage est un processus irréversible qui transforme une donnée (comme un mot de passe) en une empreinte numérique fixe. Cela signifie que même si un attaquant accède à la base de données, il ne pourra pas récupérer les mots de passe en clair.

Bcrypt a la particularité d'intégrer automatiquement un sel (salt) unique à chaque mot de passe. Le sel est une valeur aléatoire ajoutée avant le hachage, ce qui rend chaque empreinte unique, même si deux utilisateurs ont le même mot de passe. Cela empêche l'utilisation de tables de correspondance.

3.5.1.4. NaCl

J'ai utilisé la bibliothèque NaCl (*Networking and Cryptography Library*) pour assurer un chiffrement sécurisé des messages. NaCl est une bibliothèque cryptographique moderne.. Elle permet notamment le chiffrement symétrique, asymétrique, la signature de messages, ainsi que la génération de clés, avec une forte attention portée à la sécurité et à la facilité d'intégration. En Python, NaCl est disponible via le module PyNaCl.

Plus précisément, j'ai utilisé le module SecretBox de NaCl, qui permet un chiffrement symétrique authentifié. Il repose sur l'algorithme XSalsa20 (chiffrement symétrique, 192bit qui chiffre et 24 bits de nonce¹) pour le chiffrement et Poly1305 pour l'authentification, garantissant à la fois la

¹Nonce: nombre arbitraire utilisé une seule fois dans une communication cryptographique, souvent généré de manière "pseudo-aléatoire"

confidentialité et l'intégrité des messages. Le principe est simple : une clé symétrique unique est générée pour chaque message à l'aide d'un générateur de nombres aléatoires sécurisé. Cette clé est ensuite utilisée pour créer un coffre de chiffrement (SecretBox), qui va transformer le message lisible en une séquence chiffrée non interprétable sans la clé.

Dans mon API, la route `/crypt_message/` permet de chiffrer dynamiquement un message reçu via une requête HTTP. Le message est converti en bytes, chiffré, puis encodé en Base64 afin d'en faciliter le transport et le stockage. La clé utilisée est également encodée en Base64 et temporairement stockée dans un dictionnaire pour un éventuel déchiffrement. Cette approche permet de sécuriser les messages transmis tout en gardant un système flexible et évolutif.

L'utilisation de NaCl dans ce contexte garantit que chaque message est chiffré avec une clé unique, qu'un nonce aléatoire est intégré automatiquement au message, et qu'une empreinte d'authentification est jointe pour détecter toute tentative de modification. Cette combinaison de sécurité, simplicité et fiabilité fait de NaCl une solution idéale pour les applications nécessitant une forte protection des données.

3.5.1.5. Smtplib

Pour mettre en œuvre le système de notification par email dans mon application, j'ai utilisé le module Python intégré `smtplib`. Cette bibliothèque permet d'envoyer des emails en utilisant le protocole SMTP (*Simple Mail Transfer Protocol*), utilisé pour transférer les messages électroniques d'un serveur à un autre.

Smtplib offre une interface simple permettant de :

- se connecter à un serveur SMTP (comme celui de Gmail, Brevo, Outlook, etc.),
- s'authentifier avec un identifiant et un mot de passe ou un jeton sécurisé,
- composer un message (objet, contenu, destinataire),
- et l'envoyer de manière sécurisée, souvent via une connexion chiffrée (TLS ou SSL).

L'utilisation de `smtplib` dans mon projet permet donc de gérer l'envoi automatique de courriels, que ce soit pour confirmer une inscription (voir annexe) ou pour alerter l'utilisateur d'une connexion à son compte (voir annexe).

Ce système d'alerte vise à impliquer activement l'utilisateur dans la sécurité de son compte, tout en ajoutant une couche de protection contre les accès frauduleux.

3.5.5.2. *Let's encrypt*

Un certificat auto-signé n'est pas considéré comme fiable par défaut, car l'autorité de certification (CA) qui l'a émis n'est pas reconnue par les navigateurs ou les clients HTTPS standards. Cela peut entraîner des alertes de sécurité et empêcher certaines applications d'établir une connexion sécurisée. En l'occurrence dans mon projet il était impossible de communiquer avec mon API avec certificats auto-signés.

Quand je me suis rendu compte de cette limite, j'ai choisi de ne pas me reposer uniquement sur des certificats auto-signés pour sécuriser mes API. À la place, j'ai opté pour une solution plus robuste et reconnue : Let's Encrypt, une autorité de certification gratuite, automatisée et ouverte, utilisée pour chiffrer les communications sur Internet.

Pour générer et déployer ces certificats, j'ai utilisé Certbot, l'outil officiel recommandé par Let's Encrypt. Cet utilitaire m'a permis d'obtenir facilement des certificats SSL/TLS valides, puis de les configurer automatiquement `uvicorn` (un outil python qui permet de faire tourner mes API). Grâce à cette approche, mes API bénéficient d'un chiffrement robuste, tout en garantissant la confiance des clients et navigateurs.

3.5.5.3. *Domain Name Server (DNS)*

Pour générer des certificats SSL/TLS avec Let's Encrypt, il est nécessaire de prouver la propriété du nom de domaine. En effet, Let's Encrypt ne délivre des certificats que pour des noms de domaine accessibles publiquement, afin de garantir que seuls les administrateurs légitimes peuvent obtenir un certificat pour un domaine donné.

Dans mon cas, j'ai dû recourir à un DNS lié à mon nom de domaine (que je possédais déjà avant de me lancer dans le projet) car la validation effectuée par Let's Encrypt repose sur le fait que le domaine pointe bien vers le serveur pour lequel le certificat est demandé. Cette validation peut se faire de deux manières principales :

- Challenge HTTP (HTTP-01) : Let's Encrypt tente d'accéder à un fichier spécifique sur le serveur via l'URL `http://mondomaine/.well-known/acme-challenge/`. Cela nécessite que le domaine soit résolu correctement via le DNS et que le serveur web soit configuré pour répondre à la requête.
- Challenge DNS (DNS-01) : une autre méthode consiste à ajouter un enregistrement TXT dans la zone DNS du domaine. Let's Encrypt vérifie alors la présence de cet enregistrement via une requête DNS.

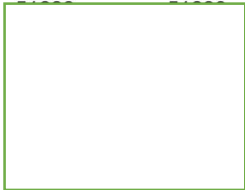
Dans les deux cas, un enregistrement DNS fonctionnel est indispensable. Sans domaine, il est impossible de valider la demande de certificat auprès de Let's Encrypt.

3.5.5.4. Redirections de ports

Pour permettre l'accès à mes APIs et mon VPN depuis l'extérieur de mon réseau local, j'ai mis en place des redirections de ports (port forwarding) sur le routeur. En effet, par défaut, les services hébergés sur une machine au sein d'un réseau local ne sont pas directement accessibles depuis Internet pour des raisons de sécurité et de protection contre les accès non autorisés.

La redirection de ports consiste à configurer le routeur afin qu'il transmette les requêtes entrantes, reçues sur des ports spécifiques, vers l'adresse IP locale et le port de la machine hébergeant les APIs. Par exemple, si une requête externe arrive sur le port 443 du routeur, celui-ci la redirige automatiquement vers le port 443 de mon serveur API.

Grâce à cette technique, les utilisateurs et applications externes peuvent accéder à mes services de manière sécurisée tout en gardant mon réseau interne isolé du réseau public. Cette étape a été essentielle pour tester et valider le fonctionnement de mes APIs en conditions réelles d'utilisation.

Votre description	Adresse IP locale	Protocole	Port local	Port public		
wireguard	192.168.0.5	TCP/UDP			 	☒
mariadb	192.168.0.13	TCP/UDP			 	☒
API	192.168.0.6	TCP			 	☒
fastapu	192.168.0.6	TCP			 	☒
API avec SSL	192.168.0.6	TCP	443	443	 	☒
						

3. Mise en place de l'infrastructure distante

Une fois l'infrastructure locale configurée, je me suis attaqué au déploiement de l'application sur l'infrastructure distante. Cette étape a consisté à préparer l'environnement de production, à transférer les fichiers nécessaires et à m'assurer du bon fonctionnement de l'application à distance. J'ai également travaillé sur la mise en forme de l'interface pour garantir une expérience utilisateur fluide et intuitive.

3.5. Flutter

Flutter est un SDK (kit de développement logiciel) open source proposé par Google, permettant de créer des applications multiplateformes à partir d'un même code source, et donc compatibles avec les systèmes d'exploitation Android, iOS, web, Windows, macOS et Linux. Flutter est fondé sur le langage Dart également développé par Google qui se distingue par sa rapidité de compilation et sa syntaxe moderne.

L'un des principaux avantages de Flutter est la performance de son moteur de rendu graphique permettant de créer des interfaces utilisateurs à la fois fluides, plaisantes à l'œil et très personnalisables. Le système de widgets de Flutter permet de construire des interfaces utilisateurs complexes tout en garantissant une grande homogénéité visuelle et une haute performance.

Pour la réalisation de mon application, j'ai choisi Flutter à la fois pour sa souplesse, son écosystème riche et vivant et sa capacité à s'intégrer sans difficulté dans de nombreux outils modernes de développement

3.6. Interface graphique

L'interface graphique de mon application est composée de plusieurs écrans, notamment l'interface de connexion, l'interface d'inscription et l'interface utilisateur. Les captures d'écran de ces écrans sont disponibles en annexe.

- Interface de connexion
- Interface d'inscription
- Interface utilisateur
 - Liste des messages
 - Page de notifications
 - Page de paramètres(déconnexion et mentions légales)
 - Page de profil de l'utilisateur (permet le changement de mot de passe et la suppression du compte)

3.7. Android (APK)

Dans l'éventualité où certains utilisateurs souhaiteraient utiliser l'application exclusivement sur leur téléphone, une version APK a été mise en place.

3.7.5. Dropbox

Afin de conserver l'esprit d'un projet à faible coût, j'ai choisi de ne pas publier l'application sur le Google Play Store. Le fichier APK est donc hébergé sur Dropbox, ce qui permet de générer un lien de partage simple et accessible. Les utilisateurs peuvent ainsi télécharger et installer l'application en toute autonomie.

Tous les fichiers

Nexuschat @

Vous uniquement

Partager le dossier

Récents Favoris

☰

Nom ↑

Qui y a accès ?

Date de modification

Nexuschat.apk

Vous uniquement

2025-05-19 15:31

4. Ce que j'aurais aimé mettre en place

Bien que l'application soit fonctionnelle, j'aurais voulu ajouter beaucoup plus de fonctionnalités pour que mon projet soit presque identique à ce que les utilisateurs connaissent aujourd'hui comme Whatsapp par exemple.

5.2. Ajout d'un service volP

J'aurais aimé ajouter des fonctionnalités supplémentaires, telles qu'implémenter un service IP pour que les utilisateurs ne disposent pas uniquement d'une communication par messages.

J'avais trouvé un service qui pouvait répondre à ma demande : Agora était la solution gratuite que je voulais utiliser. En effet, ayant une préférence pour les documentations provenant des propres fournisseurs de services, Agora en proposait une pour une utilisation avec Flutter.

Agora était la solution qui m'aurait permis d'intégrer un service VoIP grâce à sa gratuité (ce qui reste dans la philosophie du projet) et à la documentation fournie directement par la plateforme.

5.3. possibilité d'envoyer des médias

Dans les e-mails, il est mentionné que le transfert de médias est possible. Or, je ne l'ai pas implémenté dans mon projet. En effet, je n'ai pas réussi à l'intégrer car je comptais utiliser Firebase pour stocker les fichiers et les photos, mais ce service ne propose plus d'offre gratuite. Je n'ai donc pas eu le temps de chercher une solution alternative viable.

5.4. Correction de bugs et améliorations

Une application sans bug n'existe pas. Mon projet n'en est malheureusement pas épargné. C'est pourquoi j'aurais aimé régler certaines anomalies qui subsistent encore, notamment des comportements imprévus lors de l'envoi de requêtes API ou dans la gestion des connexions réseau instables.

Faute de temps, je n'ai pas pu corriger tous ces problèmes, mais des pistes sont déjà identifiées : gestion plus fine des erreurs HTTP, ajout de délais de reconnexion automatiques, et amélioration des messages d'erreur affichés à l'écran. Ces correctifs pourront être mis en œuvre dans une future version.

5.5 Système de monitoring

L'installation d'un système de monitoring centralisé aurait représenté la solution idéale pour ce projet en permettant une supervision globale et en temps réel de l'ensemble de l'infrastructure. Il aurait fourni une vision d'ensemble complète des serveurs physiques, et tout particulièrement du Raspberry Pi, élément central de l'architecture, en offrant le suivi de son état de fonctionnement, de ses performances ainsi que de la disponibilité des API hébergées.

La surveillance des machines virtuelles aurait elle aussi largement profité d'une telle solution, qui aurait permis d'être toujours informé de leur état d'exploitation (actives, stoppées, en surcharge...), de la consommation de leurs ressources (CPU, mémoire, espace disque) et d'éventuelles alertes systèmes. La base de données, elle aussi, aurait fait l'objet d'une surveillance par le biais du système de monitoring, permettant de détecter promptement une quelconque anomalie (ralentissement, surcharge, perte de connectivité).

Un tel déploiement m'aurait permis d'avoir un système de surveillance automatique sur l'ensemble des appareils, et c'est essentiel du fait que personnellement j'étais souvent en déplacement, car j'aurais reçu des alertes en cas de dysfonctionnement, au lieu de devoir réaliser une surveillance manuelle. En cas de panne d'un service ou d'un matériel, j'aurais ainsi été dans la possibilité de mettre tout en œuvre, en toute précision, pour remédier au problème sous la disponibilité des informations nécessaires au bon diagnostic de la situation.

5. Conclusion

Le projet de fin d'études que j'ai réalisé a consisté en la conception d'une application de messagerie instantanée sécurisée que j'ai nommé Nexuschat, développée à l'aide de technologies modernes, open source et donc accessibles. J'ai ainsi dû passer par diverses étapes allant de la mise en place d'une infrastructure virtualisée avec Proxmox, à la sécurisation des échanges, soit avec SSL , la création d'une API de communication avec FastAPI, et jusqu'à la création d'une interface utilisateur avec Flutter. Chaque étape m'a permis de m'améliorer dans des domaines tels que la gestion des systèmes d'exploitation et de réseaux, le développement backend et le développement frontend, ou encore la découverte de certaines failles de sécurités. J'ai pu aussi mieux intégrer certains concepts tels que l'utilisation et le fonctionnement des API.

Mais au-delà de la technique, ce projet m'a surtout confronté à des problèmes concrets, tels que la gestion de la ressource limitée, le respect de la vie privée des données utilisateurs (Le Règlement Général sur la Protection des Données (RGPD²)), l'authentification ou encore la résilience de l'ensemble. Il m'a également permis d'accéder à une certaine compréhension des exigences d'un projet large, de sa conception jusqu'à son déploiement.

² RGPD : Règlement de l'union européenne qui vise à renforcer la protection des données à caractère personnel des citoyens de l'union européenne.

Si certaines fonctionnalités, telles que les appels VoIP ou l'envoi de fichiers, n'ont pu être finalisées, j'avais su tout de même identifier les bonnes solutions technologiques à envisager pour un futur développement. Cela démontre ma capacité à analyser, proposer et anticiper des solutions permettant d'élargir le panel fonctionnel de l'application dans le but de parvenir à une application plus complète et plus professionnelle.

En somme, ce travail représente l'aboutissement de plusieurs mois de recherches, de tests et de développement, et je suis fier d'avoir pu livrer une solution fonctionnelle, cohérente et respectueuse de la vie privée des utilisateurs.

6. Sitographie

5.1. Apache2

- *apache2 [Wiki ubuntu-fr]*. (s. d.-b). <https://doc.ubuntu-fr.org/apache2>
- *tutoriel : securiser_apache2_avec_ssl [Wiki ubuntu-fr]*. (s. d.). https://doc.ubuntu-fr.org/tutoriel/securiser_apache2_avec_ssl (consulté le 13 Février 2025)

5.2. Let's encrypt

- *Mise en place d'un certificat Let's Encrypt avec Apache sur Debian 10 - Mise en place d'un certificat Let's Encrypt avec Apache sur Debian 10*. (s. d.). https://fr-wiki.ikoula.com/fr/Mise_en_place_d%E2%80%99un_certificat_Let%E2%80%99s_Encrypt_avec_Apache_sur_Debian_10 (consulté le 5 Mai 2025)
- *CertBot Instructions*. (s. d.). Certbot. <https://certbot.eff.org/instructions?ws=apache&os=pip> (consulté le 5 Mai 2025)

5.3. MariaDB

- *MariaDB Commands cheatsheet*. (s. d.). Gist.
<https://gist.github.com/Jonasdero/b4c8a5e284e13aaf456f44f5dc60257e> (consulté le 10 Octobre 2024)
- *mariadb [Wiki ubuntu-fr]*. (s. d.). <https://doc.ubuntu-fr.org/mariadb> (consulté le 10 Octobre 2024)
- *Securing Connections for Client and Server*. (s. d.). MariaDB KnowledgeBase.
<https://mariadb.com/kb/en/securing-connections-for-client-and-server/> (consulté le 10 Octobre 2024)
- *Open Source Database (RDBMS) for the Enterprise | MariaDB*. (s. d.). MariaDB.
<https://mariadb.com/docs/server/security/data-in-transit-encryption/create-self-signed-certificates-keys-openssl/> (consulté le 10 Octobre 2025)

5.4. Openssl

- Boucheron, B., Drake, M., & Heidi, E. (2019, 22 juillet). *How To Create a Self-Signed SSL Certificate for Apache in Debian 10*. DigitalOcean.
<https://www.digitalocean.com/community/tutorials/how-to-create-a-self-signed-ssl-certificate-for-apache-in-debian-10> (consulté le 20 Mars 2025)

5.5. Proxmox

- Proxmox. (s. d.). *Downloads*. <https://www.proxmox.com/en/downloads/proxmox-virtual-environment> (consulté le 15 Octobre 2025)

5.6. Proftpd

- *proftpd [Wiki ubuntu-fr]*. (s. d.). <https://doc.ubuntu-fr.org/proftpd> (consulté le 10 Novembre 2024)

5.7. Pymysql

- *PyMySQL documentation — PyMySQL 0.7.2 documentation.* (s. d.). <https://pymysql.readthedocs.io/en/latest/> (consulté le 12 Février 2025)

5.8. python

5.8.1. FastAPI

- *Tutorial - User Guide - FastAPI.* (s. d.). <https://fastapi.tiangolo.com/tutorial/> (consulté le 12 Février 2025)

5.8.2. Bcrypt

- *bcrypt.* (2025, 28 février). PyPI. <https://pypi.org/project/bcrypt/> (consulté le 15 Février 2025)
- *npm : bcrypt.* (s. d.). Npm. <https://www.npmjs.com/package/bcrypt> (consulté le 15 Février 2025)

5.8.3. Nacl

- *PyNaCL.* (2022, 7 janvier). PyPI. <https://pypi.org/project/PyNaCl/> (consulté le 15 Février 2025)
- *base64 — Base16, Base32, Base64, Base85 Data Encodings.* (s. d.). Python Documentation. <https://docs.python.org/3/library/base64.html> (consulté le 15 Février 2025)

5.8.4. smtplib

- *Envoyer un mail par SMTP en python.* (s. d.). <https://python.doctor/page-python-envoyer-mail-smtp> (consulté le 10 Mai 2025)

5.9. Flutter

- *Flutter documentation.* (s. d.). <https://docs.flutter.dev/> (consulté le 12 Février 2025)
- *shared_preferences | Flutter package.* (s. d.). Dart Packages. (consulté le 03 Mars 2025) https://pub.dev/packages/shared_preferences
- *http | Dart package.* (s. d.). Dart Packages. <https://pub.dev/packages/http> (consulté le 03 Mars 2025)
- *adaptive_theme | Flutter package.* (s. d.). Dart Packages. https://pub.dev/packages/adaptive_theme (consulté le 15 Mars 2025)
- Maxland. (s. d.). *GitHub - maxland255/flutter_giphy : A Flutter package for interacting with the Giphy API and a full-featured GIF picker UI, enabling seamless search and selection of GIFs, stickers, and emojis.* GitHub. https://github.com/maxland255/flutter_giphy (consulté le 2 Mai 2025)

5.10. Intelligence artificielle

- OpenAI. (2025). *ChatGPT* (version GPT-4o). <https://chat.openai.com>

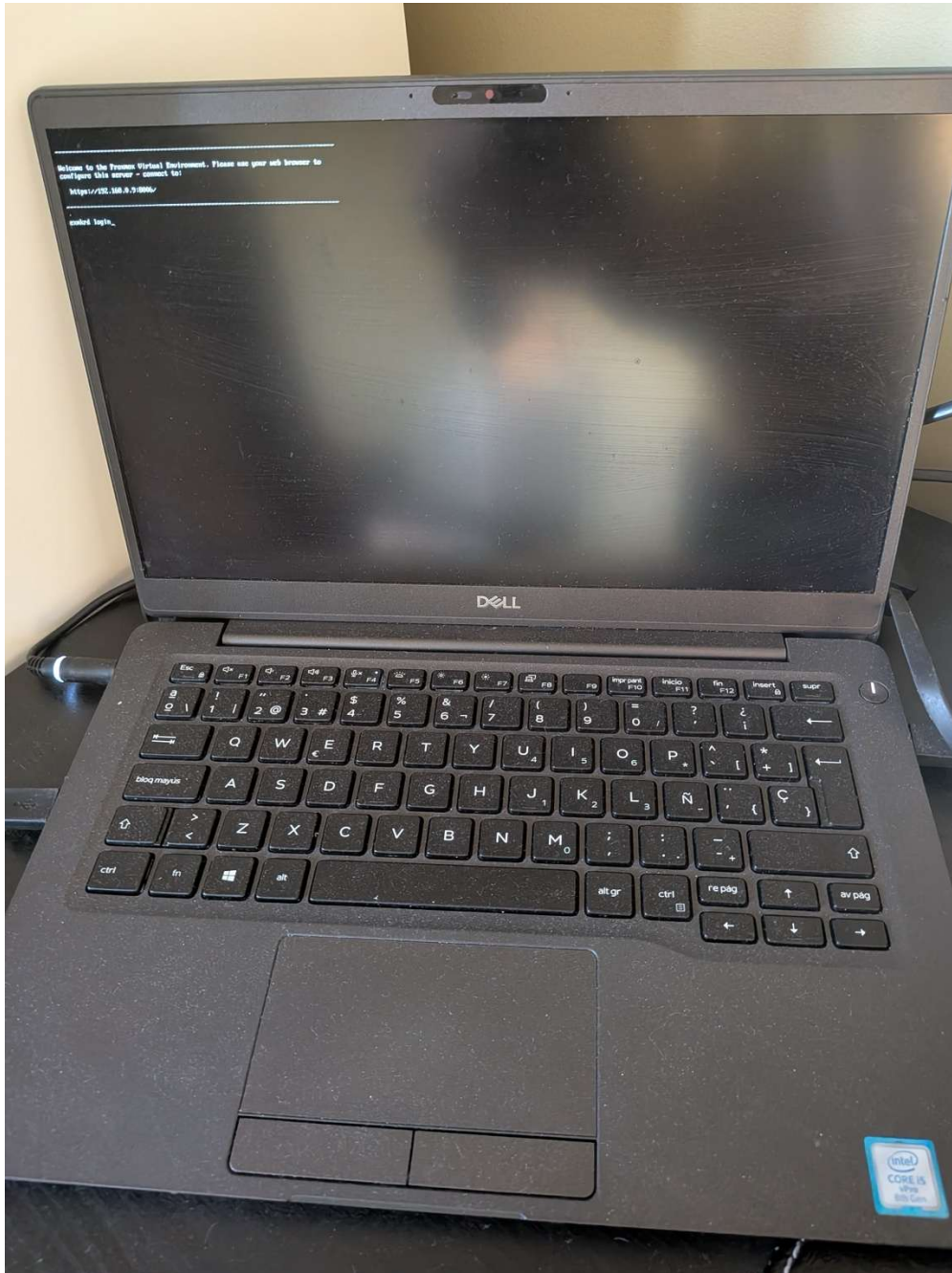
5.11. Raspberry

- *Raspberry Pi 5 8 GB Quad-Core ARMA76 (64 bits - 2,4 GHz) : Amazon.fr : CD et vinyles. (s. d.).*

https://www.amazon.fr/Raspberry-Pi-Quad-Core-ARMA76-Bits/dp/B0CK2FCG1K/ref=sr_1_7?__mk_fr_FR=%C3%85M%C3%85%C5%BD%C3%95%C3%91&crid=2EE6H89PWZGCL&dib=eyJ2IjojMSJ9.EK5CW71bdOZ_PC0v3vsz9JpWoieDSCeytda8d4WgxFx9K9ovK8yEkxhzRFFsvjjmsyZu3sxlGTYSHHYbVjv1IcdlBmom0D3WxKD5eyLVY2ZeFwJhC-T1I0UErGDBHKaq81UPC2rwvhFuFhT2uO-Ulct2johEMkIfM3o04essLI-sjIS5yWU9jIU_5Ns9t4osO1IpHaViy027gwOi2JplD5iR1IkalxJN-0VXtypOT0dCc9Z2VoNfnQdXUzUZVNNhE0Nh5RR2L1cqXCXef3Lktvr8CakXuZtKX7Dig3U9Gk.b5AAHuPIDI7ljyQCDKJwm8SISDYH-ggqJM3k7uBNYGkw&dib_tag=se&keywords=raspberry+pi+5&qid=1747924252&sprefix=raspberrypi+5%2Caps%2C92&sr=8-7

6. Annexe

6.1. Ordinateur hébergeant Proxmox



6.2. Raspberry Pi 5



6.3. Hashage de mot de passe

```
password
$2b$12$9030.uUz1ztVp5qWxTueQuaBpAaCM2i8p7lWlQkcQfsmPXor6ygmq
$2b$12$.dQy4gcZp1ZNb0PWYqeU.89bhbMV05ZsBe66PRJ0R5nifu14e0Ca
$2b$12$M4qT14PuKRRsAxtiWlomh.Zu6d3rInlyH2AwZPuLzkXpxdIYKC20u
$2b$12$fXzC09tAtLB3waThQRem5.ynyMtuPnduA7w.gjze4aUph7sjBWVyK
$2b$12$b3F8h1SI8jhURJxCabKgmuCP9.SV8SfEeHrxdPJutWwqLWKSE73/e
$2b$12$1CQ9Pw6MMk02trXUIsuC9e7ScwRA5qQi7ovtt4GY.WjhTJlq.Fz1K
$2b$12$TEUdVxsVlUDlIrb.kWaUzOWxFNGiMHJT26Js9DtuIbSHyzIBGmDQC
$2b$12$hPxUx/.tLZcZ.Ge/EEtxnOPGarWubgVgc9IKIp1yIn6.oYK0ThKBa
$2b$12$z5rJ2H5aFsBmf8IFwiDoueWJ0raSXhHP0A14Y9gr0.LRY0eIYbVDq
$2b$12$iIFmCKWPr/NzeX88s1tr6unvwj60tZkWCASnX2HBi30Qf/SsTaQBS
$2b$12$P7sfMU.drJTcmnyOAsFj50BySHTyN0WaXjk8BwRBksxT53bR.D9K2
$2b$12$VRVOYCSAfcjdVwCmzVkJZTeFEU6nR0fksKTE4nGayds9/kPVrcLmfc
```

6.4. API-FastAPI

POST	/messages/send_message/	Send Message	▼
GET	/messages/get_message/	Get Message	▼
POST	/messages/encrypt_message/	Crypt Message	▼
POST	/messages/decrypt_message/	Uncrypt Message	▼
DELETE	/messages/delete_messages/	Delete Messages	▼
POST	/messages/messages/delete_message/	Delete Message	▼
POST	/users/create_user	Create User	▼
POST	/users/delete_user	Delete User	 ▼
POST	/users/change_password	Change Password	▼
POST	/users/check_credentials	Check Credentials	▼
GET	/users/get_users	Get Users	▼
GET	/users/get_username/	Get Username	▼
GET	/users/check_email/	Check Email	▼
GET	/users/change_username/	Change Username	▼
POST	/contacts/add_contact	Add Contact	▼
POST	/contacts/add_contact	Add Contact	▼
POST	/contacts/add_contact	Add Contact	▼
POST	/contacts/add_contact	Add Contact	▼
POST	/contacts/add_contact	Add Contact	▼
POST	/contacts/demande_contact	Envoyer Demande	▼
POST	/contacts/accepter_demande	Accepter Demande	▼
POST	/contacts/refuser_demande	Refuser Demande	▼
GET	/contacts/mes_demandes	Mes Demandes	▼
GET	/contacts/mes_contacts	Mes Contacts	▼
POST	/contacts/supprimer_contact	Supprimer Contact	▼
GET	/contacts/etat_relation	Verifier Relation	 ▼
POST	/conversation/create_conv/	Create Conv	▼
POST	/conversation/delete_conv/	Delete Conv	▼
GET	/conversation/check_conv/	Check Conv	▼
GET	/conversation/get_conv/	Get Conv	▼
GET	/conversation/get_id/	Get Conv	▼
POST	/email/send_email/	Send Email	▼
GET	/email/verify_email	Verify Email	▼

6.5. Docker

Container list

Containers

Q Search... x

Start

Stop

Kill

Restart

Pause

Resume

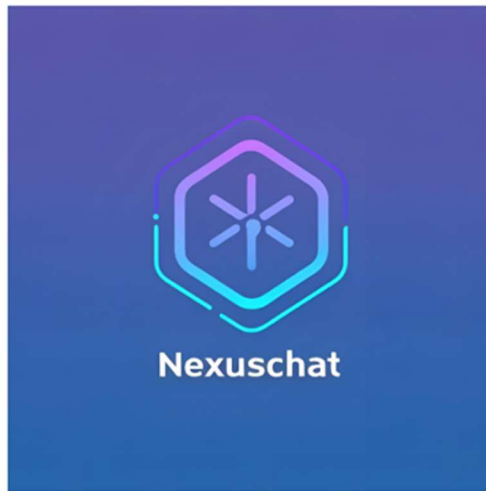
Remove

+ Add container

☐	Name	State	Quick Actions	Stack	Image	Created	IP Address	Published Ports
☐	Admin_Dashboard	running		-	nginx:latest	2025-05-14 15:56:59	172.17.0.3	-
☐	portainer	running		-	portainer/portainer-ce:latest	2025-01-15 20:39:16	172.17.0.2	8000:8000 9000:9000
☐	Wireguard	healthy		wg-easy	ghcr.io/wg-easy/wg-easy	2025-01-15 21:08:14	172.18.0.2	51820:51820 51821:51821

Items per page 10

6.6. Email de Réinitialisation de mot de passe



Changement de mot de passe

Bonjour,

Suite à votre demande de réinitialisation de mot de passe, cliquez sur le bouton ci-dessous pour définir un nouveau mot de passe :

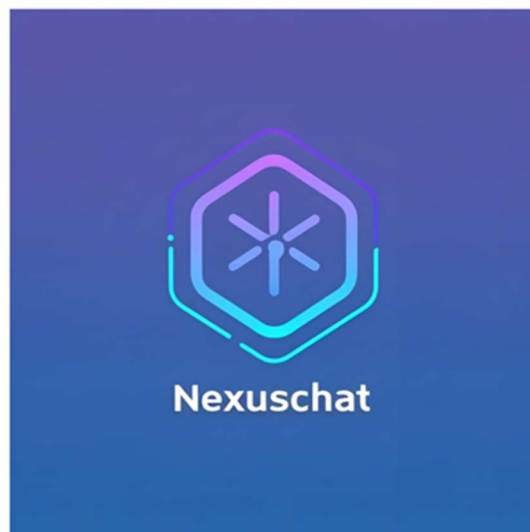
Réinitialiser mon mot de passe

Ce lien est valable pendant 24 heures. Si vous n'avez pas demandé cette action, vous pouvez ignorer ce message.

Si cette tentative est de vous, vous pouvez ignorer ce message. Sinon, pensez à sécuriser votre compte.

Cet email vous est envoyé automatiquement par NexusChat pour votre sécurité.

6.7. Email de tentative de connexion



Tentative de connexion détectée

Bonjour,

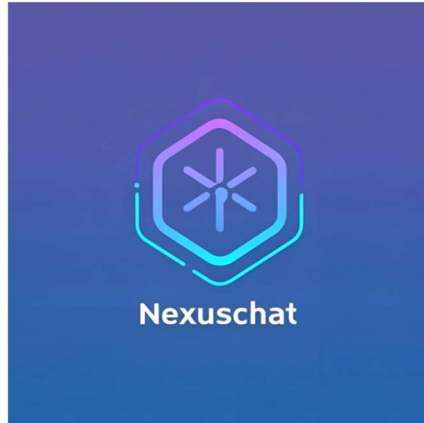
Nous avons détecté une tentative de connexion à votre compte NexusChat.

Adresse IP : [REDACTED]

Si cette tentative est de vous, vous pouvez ignorer ce message. Sinon, pensez à sécuriser votre compte.

Cet email vous est envoyé automatiquement par NexusChat pour votre sécurité.

6.8. Email de confirmation de compte



Bienvenue sur NexusChat

Merci de vous être inscrit sur NexusChat ! 🎉

Nous sommes ravis de vous accueillir dans notre communauté. Avant de commencer à discuter, nous devons vérifier votre adresse e-mail.

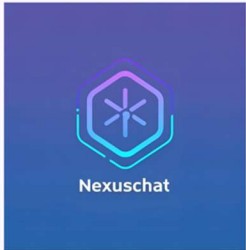
[Confirmer mon compte](#)

Ce lien expire dans 24 heures. Si vous avez besoin d'un nouveau lien, veuillez vous connecter et demander une nouvelle vérification.

Avec NexusChat, vous pourrez discuter instantanément avec vos amis, partager des médias et bien plus encore !

6.9. Interface utilisateur

6.9.1. Page de connexion



Bienvenue sur NexusChat

Adresse email

Mot de passe

Se connecter

S'inscrire

6.9.2. Page d'inscription



The image shows a registration form titled "S'inscrire sur NexusChat". It is contained within a light purple rectangular box. The form has three input fields: "Nom d'utilisateur", "Adresse email", and "Mot de passe". Below these fields are two buttons: "S'inscrire" and "Déjà inscrit ?".

S'inscrire sur NexusChat

Nom d'utilisateur

Adresse email

Mot de passe

S'inscrire

Déjà inscrit ?

6.9.3. Liste des messages

Bienvenue !

Mes Conversations

Mmaxime

MMaxland255

Chat

Notifications

Paramètres

6.9.4. Profil de l'utilisateur

[←](#)

Profil



Nom d'utilisateur

parotux

Email

✓ maxime.derick1235478@gmail.com

Changer nom d'utilisateur

Changer le mot de passe

Supprimer le compte

6.9.5. Page de contact (liste de contacts de l'utilisateur)

←

Mes contacts

Contacts

Demandes

Rechercher

Q

Rechercher dans mes contacts...

maxime

Chat

maxime2

Chat

Maxland255

Chat

6.9.6. Page de contact section recherche

←

Mes contacts

Contacts

Demandes

Rechercher

Q

Rechercher un utilisateur...

Personnes que tu pourrais connaître

Ismail

Demande en attente

Ben Sellam

Demande en attente

6.9.7. Page de notifications

Bienvenue !

Notifications

text
test
19 May 2025, 15:33



Chat

Notifications

Paramètres

6.10. Interface administrateur



NexusChat Admin

Utilisateurs

Liste des utilisateurs

ID	Expediteur	Destinataire	Heure d'envoi	Actions	
12	maxime	derick.maxime@derickexm.be	2025-03-12	Modifier	Supprimer
15	parotux	maxime.derick1235478@gmail.com	2025-04-04	Modifier	Supprimer
17	Martin	Trompette1200@gmail.com	2025-04-08	Modifier	Supprimer
18	Ben Sellam	abdelkader.bensellam@inraci.be	2025-04-08	Modifier	Supprimer
19	1300	quentinclaes@icloud.com	2025-04-10	Modifier	Supprimer
20	Ismail	koksai.ismail10@gmail.com	2025-04-11	Modifier	Supprimer
23	alexiscento	alexis99zig@gmail.com	2025-04-15	Modifier	Supprimer
28	Necry	azappapapappapap12@gmail.com	2025-04-17	Modifier	Supprimer
29	anonyme	xrgameur4268551@gmail.com	2025-04-21	Modifier	Supprimer
38	Maxland255	harrypiet07@gmail.com	2025-04-23	Modifier	Supprimer
39	test	test@gmail.com	2025-05-02	Modifier	Supprimer
40	maxime2	maxime2rick@gmail.com	2025-05-20	Modifier	Supprimer